Logical Reasoning in the Mafia Game

In this assignment, logical reasoning is applied to the classic Mafia game in order to formally determine the outcome of the night phase. Instead of resolving night actions purely through procedural code, a first-order logic theorem prover (Prover9) is used to validate whether a player must die, based on the roles and actions chosen during the night.

The goal of this approach is to demonstrate how symbolic logic can be used to model game rules declaratively and ensure correctness in complex multi-agent interactions.

Roles and Game Knowledge

The Mafia game includes the following roles:

- **Killer**: chooses a target to kill during the night.
- **Doctor**: chooses a player to protect.
- **LadyCcompanion**: chooses a player to block, preventing their action.
- **Cop**: investigates a player and privately learns whether they are the Killer.
- **Villager**: has no night action.

At the beginning of the game, roles are randomly assigned and remain hidden, except for the human player's own role. During gameplay, only public outcomes (such as deaths) are revealed.

Night Phase Logic

During the night phase, all alive players may perform an action depending on their role. These actions are collected into a single structure mapping each player to a chosen target (or `None` if no action is performed).

The night resolution must correctly handle multiple interacting effects:

- Blocking by the LadyCompanion
- Protection by the Doctor
- Killing by the Killer
- The absence of actions (e.g., Villagers)

Because these effects interact non-trivially, the outcome is resolved using formal logic.

Logical Predicates

The following predicates are used in the Prover9 model:

- `Role(x,r)` – player `x` has role `r`
- `Target(x,y)` – player `x` targets player `y`
- `Blocked(y)` – player `y` is blocked
- `Protected(y)` – player `y` is protected
- `Dies(y)` – player `y` dies during the night

Logical Rules

The Mafia rules are encoded using first-order logic equivalences:

Blocking Rule

A player is blocked if and only if the LadyCcompanion targets them:

$$Blocked(y) \leftrightarrow \exists x \, (Role(x, LadyCompanion) \land Target(x, y))$$

Protection Rule

A player is protected if the Doctor targets them and the Doctor is not blocked:

$$Protected(y) \leftrightarrow \exists x \, (Role(x, Doctor) \land Target(x, y) \land \neg Blocked(x))$$

Death Rule

A player dies if the Killer targets them, the Killer is not blocked, and the target is not protected:

$$Dies(y) \leftrightarrow \exists x \, (Role(x, Killer) \land Target(x, y) \land \neg Blocked(x) \land \neg Protected(y))$$

Ensuring Logical Correctness

To avoid incorrect inferences, the implementation enforces:

- All players are distinct individuals.
- All roles are distinct and exclusive.
- `Role` and `Target` predicates are defined using biconditional (`<->`) expressions, preventing Prover9 from inventing additional roles or actions.

Use of Prover9

For each night:

1. The game builds a Prover9 input file containing all assumptions (roles, targets, rules).
2. A goal is specified:
   - Either `Dies(player)` for the expected victim
   - Or `all x -Dies(x)` for a peaceful night
3. Prover9 attempts to prove the goal.

Why Prover9 Matters

Using Prover9 provides several advantages:

- The night resolution is **formally correct**, not just procedurally implemented.
- Complex interactions (block vs protect vs kill) are handled cleanly.
- The rules are transparent, declarative, and easy to verify.
- The system scales better as new roles or rules are added.

A2 Original Code Snippets (Mafia Game)

Game Initialization

```
def start_game(self):
    self.log("=== NEW GAME STARTED ===")
    role_deck = ["Killer", "Doctor", "LadyCompanion", "Cop", "Villager"]
    random.shuffle(role_deck)

    for i, p in enumerate(self.players):
        self.roles[p] = role_deck[i]
        self.status[p] = "Alive"

    self.update_status_board()
    self.log(f"You are {self.player_name}. Your role is: {self.roles[self.player_name]}"
    self.start_night_phase()
```

Night AI Action Generation

```
def process_night_turn(self, player_target="None"):
    night_actions = {}
    alive = self.get_alive_players()

    for p in alive:
        if p == self.player_name:
            night_actions[p] = player_target if self.roles[p] != "Villager" else "None"
            continue

        role = self.roles[p]
        others = [x for x in alive if x != p]

        if role == "Killer":
            night_actions[p] = random.choice(others)
        elif role == "Doctor":
            night_actions[p] = p if random.random() < 0.2 else random.choice(others)
        elif role == "LadyCompanion":
            night_actions[p] = random.choice(others)
        elif role == "Cop":
            night_actions[p] = random.choice(others)
        else:
            night_actions[p] = "None"

    thread = threading.Thread(target=self.run_prover9_logic, args=(self.roles, night_act
    thread.start()
```

Win Conditions

```
def check_game_over(self):
    alive = self.get_alive_players()
    mafia_count = sum(1 for p in alive if self.roles[p] == "Killer")
    villager_count = len(alive) - mafia_count
```

```
if mafia_count == 0:
    messagebox.showinfo("Game Over", "TOWN WINS! The Killer is dead.")
    return True
elif mafia_count >= villager_count:
    messagebox.showinfo("Game Over", "MAFIA WINS! They have taken over the town.")
    return True
return False
```